CHALLENGE SHEET

Duckvault Web Ctf

A document management API with a broken object-level authorization check.

DIFFICULTY	TRACK	HOST PORT	LAB ID
EASY	WEB	8000	duckvault-web-ctf

EASY MEDIUM HARD INSANE

VERIFY THE SOLVE

```
• python3 ../scripts/check.py duckvault-web-ctf
```

Brief

DuckVault is used internally to manage company documents. You have been provided with valid employee credentials and access to the REST API. Investigate the available API endpoints and determine whether document access controls properly restrict users to their own documents. The challenge service is available locally at:

```
    Host: 127.0.0.1
    Host port: 8000
    Container port: 8000
```

The service uses REST and is not exposed through GraphQL or gRPC.

Credentials

```
    Username: alice
    Password: AliceVault2026!
```

Goal

Retrieve the challenge flag by exploiting the intended vulnerability in the DuckVault API. The flag follows this format:

```
    duck\{...\}
```

Difficulty

Easy This challenge requires multiple steps, including API enumeration, authentication, object identification, authorization testing, and exploitation.

Prerequisites

- Basic knowledge of HTTP and REST APIs
- Basic knowledge of API authentication
- Familiarity with authorization concepts
- Familiarity with a REST client such as Burp Suite or `curl`

Setup

Start the challenge from the lab directory:

```
docker compose up --build
```

The REST API will be available at:

```
Host: 127.0.0.1
Host port: 8000
```

You can verify that the service is running with:

```
curl http://127.0.0.1:8000/health
```

Reset

To reset the challenge environment:

```
docker compose down --remove-orphans
docker compose up --build
```

Rules

- Perform testing only against the provided local challenge environment.
- Do not target external systems or services.
- Do not modify the challenge source code while solving it.

Flag Format

```
duck\{...\}
```